

Designing a Student Record Management System

MODULE 1 – TEACHER LOGIN & DASHBOARD

Code

```
#include <stdio.h>
#include <string.h>
void teacherLogin()
{
    char username[20];
    char password[20];
    printf("=====\n");
    printf("    STUDENT RECORD MANAGEMENT\n");
    printf("=====\n");
    printf("\n    TEACHER LOGIN\n");
    printf("-----\n");
    printf("Username: ");
    scanf("%19s", username);
    printf("Password: ");
    scanf("%19s", password);
    if(strcmp(username, "teacher") != 0 ||
        strcmp(password, "1234") != 0)
    {
        printf("\nInvalid Username or Password!\n");
        return;
    }
    printf("\nLogin Successful!\n");
}
void displayDashboard()
{
```

```

printf("\n=====\\n");
printf("      TEACHER DASHBOARD\\n");
printf("=====\\n");

printf("1. Add Student Record\\n");
printf("2. Search Student\\n");
printf("3. Display All Students\\n");
printf("4. Sort by Average\\n");
printf("5. Save Records\\n");
printf("6. Logout\\n");
printf("7. Exit\\n");
}

```

Pseudocode

MODULE 1: TEACHER LOGIN AND DASHBOARD

FUNCTION teacherLogin

 INPUT username

 INPUT password

 IF username is "teacher" AND password is "1234"

 DISPLAY "Login Successful"

 ELSE

 DISPLAY "Invalid Username or Password"

 END IF

END FUNCTION

FUNCTION displayDashboard

 DISPLAY Teacher Dashboard

 DISPLAY Add Student

 DISPLAY Search Student

 DISPLAY Display All Students

 DISPLAY Sort Students

 DISPLAY Save Records

```
    DISPLAY Logout
    DISPLAY Exit
END FUNCTION
```

MODULE 2 – STUDENT RECORD MANAGEMENT

Code

```
#define MAX_STUDENTS 100

struct Student
{
    int rollNo;
    char name[50];
    float mark1;
    float mark2;
    float mark3;
    float total;
    float average;
};

void calculateResult(struct Student *s)
{
    s->total = s->mark1 + s->mark2 + s->mark3;
    s->average = s->total / 3.0;
}

int isDuplicateRoll(struct Student students[],
                    int count,
                    int rollNo)
{
    int i;
    for(i = 0; i < count; i++)
    {
        if(students[i].rollNo == rollNo)
```

```

        {
            return 1;
        }
    }
return 0;
}

float readMark(int subjectNumber)
{
    float mark;
while(1)
    {
        printf("Enter Mark %d (0-100): ", subjectNumber);
        scanf("%f", &mark);
if(mark >= 0 && mark <= 100)
        {
            return mark;
        }
        printf("Invalid mark! Please enter a value between 0 and 100.\n");
    }
}

void addStudent(struct Student students[], int *count)
{
    struct Student *s;
if(*count >= MAX_STUDENTS)
    {
        printf("\nError: Student record limit reached!\n");
        return;
    }
s = &students[*count];
printf("\n===== ADD STUDENT =====\n");
printf("Enter Roll Number: ");
    scanf("%d", &s->rollNo);

```

```

if(isDuplicateRoll(students, *count, s->rollNo))
{
    printf("\nError: Roll number already exists!\n");
    return;
}
printf("Enter Name: ");
scanf(" %49[^\n]", s->name);
s->mark1 = readMark(1);
s->mark2 = readMark(2);
s->mark3 = readMark(3);
calculateResult(s);
(*count)++;
printf("\nStudent record added successfully!\n");
printf("Total   : %.2f\n", s->total);
printf("Average : %.2f\n", s->average);
}

void displayStudents(struct Student students[], int count)
{
    int i;
    if(count == 0)
    {
        printf("\nNo student records available.\n");
        return;
    }

    printf("\n===== STUDENT RECORDS =====\n");

    for(i = 0; i < count; i++)
    {
        printf("\nRoll Number : %d", students[i].rollNo);
        printf("\nName       : %s", students[i].name);
        printf("\nMark 1    : %.2f", students[i].mark1);
    }
}

```

```
printf("\nMark 2    : %.2f", students[i].mark2);
printf("\nMark 3    : %.2f", students[i].mark3);
printf("\nTotal      : %.2f", students[i].total);
printf("\nAverage    : %.2f\n", students[i].average); }}
```

Pseudocode

MODULE 2: STUDENT RECORD MANAGEMENT

DEFINE Student structure

roll number

name

three subject marks

total

average

END STRUCTURE

FUNCTION calculateResult

CALCULATE total

CALCULATE average

END FUNCTION

FUNCTION isDuplicateRoll

CHECK all existing roll numbers

IF matching roll number exists

RETURN true

ELSE

RETURN false

END FUNCTION

FUNCTION readMark

REPEAT

INPUT mark

IF mark is between 0 and 100

RETURN mark

ELSE

DISPLAY invalid mark

```
        END IF
    UNTIL valid mark
END FUNCTION

FUNCTION addStudent
    CHECK maximum record limit
    INPUT roll number
    CHECK duplicate roll number
    INPUT name
    INPUT three valid marks
    CALL calculateResult
    INCREMENT record count
    DISPLAY total and average
END FUNCTION

FUNCTION displayStudents
    IF no records
        DISPLAY no records
    ELSE
        DISPLAY all student records
    END IF
END FUNCTION
```

MODULE 3 – SEARCHING & SORTING

Code

```
void searchStudent(struct Student students[], int count)
{
    int rollNo;
    int i;
    int found = 0;
    if(count == 0)
    {
        printf("\nNo student records available.\n");
        return;
    }
    printf("\n===== SEARCH STUDENT =====\n");
    printf("Enter Roll Number to search: ");
    scanf("%d", &rollNo);
    for(i = 0; i < count; i++)
    {
        if(students[i].rollNo == rollNo)
        {
            printf("\nStudent Found!\n");
            printf("-----\n");
            printf("Roll Number : %d\n", students[i].rollNo);
            printf("Name      : %s\n", students[i].name);
            printf("Mark 1   : %.2f\n", students[i].mark1);
            printf("Mark 2   : %.2f\n", students[i].mark2);
            printf("Mark 3   : %.2f\n", students[i].mark3);
            printf("Total    : %.2f\n", students[i].total);
            printf("Average  : %.2f\n", students[i].average);
            found = 1;
            break;
        }
    }
```

```
    } if(found == 0)
    {
        printf("\nStudent with Roll Number %d not found.\n",
            rollNo);
    }
}
```

```
void sortStudents(struct Student students[], int count)
```

```
{
    int i;
    int j;
    struct Student temp;

    if(count == 0)
    {
        printf("\nNo student records available for sorting.\n");
        return;
    }

    for(i = 0; i < count - 1; i++)
    {
        for(j = i + 1; j < count; j++)
        {
            if(students[i].average < students[j].average)
            {
                temp = students[i];
                students[i] = students[j];
                students[j] = temp;
            }
        }
    }
}
```

```
    printf("\nStudents sorted by average in descending order.\n");  
displayStudents(students, count);  
}
```

Pseudocode

MODULE 3: SEARCHING AND SORTING

FUNCTION searchStudent

IF no records exist

 DISPLAY no records

 RETURN

END IF

INPUT roll number

FOR each student

 IF student roll number matches

 DISPLAY student details

 SET found = true

 STOP SEARCH

 END IF

END FOR

IF found is false

 DISPLAY student not found

END IF

END FUNCTION

FUNCTION sortStudents

IF no records exist

 DISPLAY no records

 RETURN

END IF

FOR i from 0 to count - 1

 FOR j from i + 1 to count

 IF average of student i is less than

 average of student j

 SWAP the two students

```
        END IF
    END FOR
END FOR
DISPLAY students in descending order
END FUNCTION
```

MODULE 4 – FILE HANDLING & STORAGE CLASS

Code

```
int recordCounter(int operation, int value)
{
    static int count = 0;

    if(operation == 1)
    {
        count++;
    }
    else if(operation == 2)
    {
        count = value;
    }
    else if(operation == 3)
    {
        count = 0;
    }

    return count;
}

void saveRecords(struct Student students[], int count)
{
    FILE *fp;
    int i;
```

```
fp = fopen("students.txt", "w");
```

```
if(fp == NULL)
```

```
{
```

```
    printf("\nError: Unable to open file for saving!\n");
```

```
    return;
```

```
}
```

```
for(i = 0; i < count; i++)
```

```
{
```

```
    fprintf(fp, "%d|%s|%.2f|%.2f|%.2f|%.2f|%.2f\n",
```

```
        students[i].rollNo,
```

```
        students[i].name,
```

```
        students[i].mark1,
```

```
        students[i].mark2,
```

```
        students[i].mark3,
```

```
        students[i].total,
```

```
        students[i].average);
```

```
}
```

```
fclose(fp);
```

```
printf("\nRecords saved successfully to students.txt\n");
```

```
}
```

```
void loadRecords(struct Student students[], int *count)
```

```
{
```

```
    FILE *fp;
```

```
    fp = fopen("students.txt", "r");
```

```

if(fp == NULL)
{
    printf("\nNo previous record file found.\n");
    printf("Starting with empty student records.\n");
    return;
}

*count = 0;

while(*count < MAX_STUDENTS &&
    fscanf(fp, "%d|%49[^\n]|%f%f%f%f%f",
        &students[*count].rollNo,
        students[*count].name,
        &students[*count].mark1,
        &students[*count].mark2,
        &students[*count].mark3,
        &students[*count].total,
        &students[*count].average) == 7)
{
    (*count)++;
}

fclose(fp);

recordCounter(2, *count);

printf("\n%d student record(s) loaded successfully.\n",
    *count);
}

```

Pseudocode

MODULE 4: FILE HANDLING AND STORAGE CLASS

FUNCTION recordCounter

 DECLARE static count

 IF operation is increment

 INCREMENT count

 ELSE IF operation is set

 SET count to given value

 ELSE IF operation is reset

 SET count to zero

 END IF

 RETURN count

END FUNCTION

FUNCTION saveRecords

 OPEN students.txt in write mode

 IF file cannot be opened

 DISPLAY file error

 RETURN

 END IF

 FOR each student

 WRITE student details to file

 END FOR

 CLOSE file

 DISPLAY save successful

END FUNCTION

FUNCTION loadRecords

 OPEN students.txt in read mode

 IF file does not exist

 DISPLAY no previous records

 RETURN

 END IF

 SET record count to zero

 WHILE records are available

 READ student details

 INCREMENT record count

 END WHILE

 CLOSE file

 SET static counter to current record count

 DISPLAY number of loaded records

END FUNCTION

ALGORITHM – STUDENT RECORD MANAGEMENT SYSTEM

Step 1: Start the program.

Step 2: Define the Student structure with the following fields:

- Roll Number
- Name
- Mark 1
- Mark 2
- Mark 3
- Total
- Average

Step 3: Define the maximum number of student records as 100.

Step 4: Declare a student array and initialize the record count to zero.

Step 5: Display the **Teacher Login** screen.

Step 6: Read the username and password from the user.

Step 7: Check the login credentials.

- If username is teacher and password is 1234, display **Login Successful**.
- Otherwise, display **Invalid Username or Password** and terminate the program.

Step 8: After successful login, load previously saved student records from students.txt.

- If the file exists, read and store the records.
- If the file does not exist, start with empty records.

Step 9: Display the **Teacher Dashboard** with the following options:

1. Add Student Record
2. Search Student
3. Display All Students
4. Sort by Average
5. Save Records
6. Logout
7. Exit

Step 10: Read the user's dashboard choice.

If the user selects 1 – Add Student Record

Step 11: Check whether the maximum record limit of 100 has been reached.

Step 12: Read the student's roll number.

Step 13: Check whether the roll number already exists.

- If it exists, display **Roll number already exists**.
- Otherwise, continue.

Step 14: Read the student's name.

Step 15: Read the three subject marks.

Step 16: Validate each mark.

- If the mark is between 0 and 100, accept it.
- Otherwise, display an error message and ask for the mark again.

Step 17: Pass the student structure to calculateResult() using a pointer.

Step 18: Calculate:

- $\text{Total} = \text{Mark1} + \text{Mark2} + \text{Mark3}$
- $\text{Average} = \text{Total} / 3$

Step 19: Increase the student record count.

Step 20: Update the static record counter.

Step 21: Display the student record, total, and average.

If the user selects 2 – Search Student

Step 22: Check whether any student records exist.

Step 23: If records exist, input the roll number to search.

Step 24: Perform a **linear search** through the student array.

Step 25: If the roll number matches, display the complete student details.

Step 26: If no matching roll number is found, display **Student not found**.

If the user selects 3 – Display All Students

Step 27: Check whether student records exist.

Step 28: If records exist, display the details of all students.

Step 29: If no records exist, display **No student records available**.

If the user selects 4 – Sort by Average

Step 30: Check whether student records exist.

Step 31: Compare the average marks of the students.

Step 32: Swap student records when the first student's average is smaller than the second student's average.

Step 33: Continue the comparison until all records are arranged in **descending order of average**.

Step 34: Display the sorted student records.

If the user selects 5 – Save Records

Step 35: Open students.txt in write mode.

Step 36: Check whether the file was opened successfully.

- If not, display a file error message.
- Otherwise, continue.

Step 37: Write all student records into the file.

Step 38: Close the file.

Step 39: Display **Records saved successfully**.

If the user selects 6 – Logout

Step 40: Display **Logged out successfully**.

Step 41: End the dashboard loop and terminate the program.

If the user selects 7 – Exit

Step 42: Display **Thank you for using the system**.

Step 43: Terminate the program.

If the user enters any other choice

Step 44: Display **Invalid choice! Please try again**.

Step 45: Return to the Teacher Dashboard.

Step 46: Stop the program.

Problem Statement:

A college intends to modernize its student record-keeping process by replacing the traditional paper-based system with a simple, file-persistent C application capable of managing academic records efficiently. The task is to design and implement a complete C program that represents each student's data using a structure containing the roll number, name, marks in three subjects, and computed total and average. The program must include a pointer-based function, without relying on any global variables, to calculate each student's total and average marks. Using this structure, the application should allow the user to add new student records, search for a specific student by roll number using an appropriate linear search technique, and sort and display all student records in descending order of average marks. The system must also support saving all student records permanently to a file and reading and displaying these records from the file whenever the program is restarted, ensuring that data persists across multiple executions. As part of the design justification, the student must select and explain an appropriate storage class for a counter variable that tracks the number of records across multiple function calls, describing what would go wrong if a different storage class were used instead. Additionally, the design must be critically evaluated to identify at least one potential runtime error the program could encounter—such as a missing file, a duplicate roll number, or an array overflow—and the code must be modified to handle this error gracefully. The final submission should consist of complete, compiled, and tested C source code, accompanied by a short half-page write-up justifying the design decisions made for the storage class selection and the runtime error handling strategy.

IMPLEMENTATION

C:\Users\poopa\Desktop\Student_Record_Management_System\Student_Record_Management_System

```
=====
STUDENT RECORD MANAGEMENT
=====
```

TEACHER LOGIN

```
-----
Username: teacher
Password: 1234
```

Login Successful!

1 student record(s) loaded successfully.

```
=====
TEACHER DASHBOARD
=====
```

1. Add Student Record
2. Search Student
3. Display All Students
4. Sort by Average
5. Save Records
6. Logout
7. Exit

Enter your choice: 1

```
===== ADD STUDENT =====
```

```
Enter Roll Number: 102
Enter Name: priya
Enter Mark 1 (0-100): 80
Enter Mark 2 (0-100): 90
Enter Mark 3 (0-100): 95
```

```
Student record added successfully!
Total   : 265.00
Average : 88.33
```

Enter your choice: 4

Students sorted by average in descending order.

===== STUDENT RECORDS =====

Roll Number : 102
Name : priya
Mark 1 : 80.00
Mark 2 : 90.00
Mark 3 : 95.00
Total : 265.00
Average : 88.33

Roll Number : 101
Name : arun
Mark 1 : 80.00
Mark 2 : 90.00
Mark 3 : 85.00
Total : 255.00
Average : 85.00

===== TEACHER DASHBOARD =====

1. Add Student Record
2. Search Student
3. Display All Students
4. Sort by Average
5. Save Records
6. Logout
7. Exit

Enter your choice: 6

Logged out successfully.

Process returned 0 (0x0) execution time : 29.482 s
Press any key to continue.

Enter your choice: 2

===== SEARCH STUDENT =====

Enter Roll Number to search: 101

Student Found!

Roll Number : 101
Name : arun
Mark 1 : 80.00
Mark 2 : 90.00
Mark 3 : 85.00
Total : 255.00
Average : 85.00

===== TEACHER DASHBOARD =====

1. Add Student Record
2. Search Student
3. Display All Students
4. Sort by Average
5. Save Records
6. Logout
7. Exit

Enter your choice: 3

===== STUDENT RECORDS =====

Roll Number : 101
Name : arun
Mark 1 : 80.00
Mark 2 : 90.00
Mark 3 : 85.00
Total : 255.00
Average : 85.00

Roll Number : 102
Name : priya
Mark 1 : 80.00
Mark 2 : 90.00
Mark 3 : 95.00
Total : 265.00
Average : 88.33

=====

Test Case ID	Test Case Description	Input / Action	Expected Output	Actual Output	Status
TC01	Valid Teacher Login	Username: teacher Password: 1234	Login successful and Teacher Dashboard displayed	Login successful and Teacher Dashboard displayed	PASS
TC02	Invalid Teacher Login	Username: abc Password: 9999	Display "Invalid Username or Password"	"Invalid Username or Password" displayed	PASS
TC03	Add Student Record	Roll No: 101 Name: Arun Kumar Marks: 80, 90, 85	Record added; Total = 255; Average = 85	Record added; Total = 255; Average = 85	PASS
TC04	Duplicate Roll Number	Enter existing Roll No: 101	Display "Roll number already exists"	"Roll number already exists" displayed	PASS
TC05	Search Existing Student	Search Roll No: 101	Display complete details of student 101	Student 101 details displayed correctly	PASS
TC06	Search Non-Existing Student	Search Roll No: 999	Display "Student not found"	Student with Roll No 999 not found	PASS
TC07	Sort Students by Average	Select Sort option	Students displayed in descending order of average	Students sorted in descending order	PASS
TC08	Save Student Records	Select Save Records option	Records saved successfully to students.txt	Records saved successfully	PASS
TC09	Load Records After Restart	Restart program and login	Previously saved records loaded automatically	Saved records loaded successfully	PASS
TC10	Invalid Mark	Enter Mark: 150	Display invalid mark message and request valid mark	Invalid mark message displayed	PASS

Test Case ID	Test Case Description	Input / Action	Expected Output	Actual Output	Status
TC11	Invalid	Enter choice: 9	Display "Invalid	Invalid choice	PASS
	Dashboard Choice		choice! Please try again."	message displayed	
TC12	Empty Student Records	Select Search/Display/Sort before adding records	Display appropriate "No student records available" message	Appropriate message displayed	PASS

